

When Retrieval Helps: Comparing RAG Strategies for Malicious PyPI Package Detection

Trần Hồng Nhật
nhat.th261124M@sis.hust.edu.vn
HUST
Hanoi, Vietnam

Nguyễn Tùng Lương
luong.nt252743M@sis.hust.edu.vn
HUST
Hanoi, Vietnam

Nguyễn Lâm Thái
thai.nl261068M@sis.hust.edu.vn
HUST
Hanoi, Vietnam

Nguyễn Trọng Tuấn
tuan.nt261067M@sis.hust.edu.vn
HUST
Hanoi, Vietnam

Vương Đình Minh
minh.vd261073M@sis.hust.edu.vn
HUST
Hanoi, Vietnam

Abstract

Large language models (LLMs) can inspect source code for malicious behavior, but they miss subtle attacks such as obfuscated droppers and typosquats whose installation scripts look innocuous. Retrieval-augmented generation (RAG) has been proposed as the remedy, yet prior work couples retrieval to particular knowledge sources and mechanisms, leaving it unclear how much of the benefit comes from retrieval itself and how much from how retrieval is done. We isolate retrieval strategy through a controlled comparison of seven pipelines—a zero-shot baseline and six RAG variants (dense, sparse BM25, hybrid fusion, behavior-summary, contrastive dual-pool, and their combination)—using the same LLM backbone, prompting, and code-only knowledge base throughout, and evaluate on 1,020 test packages. We find that code-level retrieval improves detection on this benchmark, driven by a large recall gain; simple sparse retrieval performs comparably to a 4B dense retriever; contrastive retrieval reduces false alarms; and behavior-summary queries underperform the no-retrieval baseline. We also analyze the samples that no method detects (metadata-cloned typosquats) and use a static-grounding proxy to measure explanation faithfulness, suggesting that retrieval is associated with fewer unsupported behavior claims. As a side exploration, we also try Jev, a typed-decision model, as an external no-retrieval comparator.

CCS Concepts

• **Software and its engineering** → **Software development techniques**; • **Security and privacy** → *Systems security*.

Keywords

supply chain, malicious package, PyPI, large language model, RAG, retrieval, typosquatting

1 Introduction

Open-source package registries have become a cornerstone of modern software development, giving developers access to millions of reusable components [5]. The same accessibility, however, makes these ecosystems an attractive target for supply-chain attacks: adversaries inject malicious code into widely used libraries or publish deceptive packages that imitate legitimate ones [7, 8, 13]. Unlike traditional vulnerabilities, which stem

from inadvertent errors, malicious packages are deliberately engineered to deceive users, which makes them particularly difficult to detect [9]. Empirical studies show that such packages persist in registries despite existing safeguards [7, 13, 14], and the continued growth of the Python ecosystem keeps expanding the attack surface.

Automated detection of malicious packages has traditionally relied on static and dynamic analysis and on machine-learning classifiers over hand-crafted features [17]. More recently, large language models (LLMs) have been applied to security-oriented code analysis, including malicious package detection [9, 22]. LLMs can reason over source code in natural language and surface subtle indicators of malicious behavior, yet they are also prone to hallucination and to missing attacks whose code looks innocuous [9, 19].

Retrieval-augmented generation (RAG) has been proposed as a general remedy for such LLM shortcomings: before answering, the model conditions on documents retrieved from an external corpus [6, 12]. Within security, RAG has been used to detect vulnerabilities in smart contracts [21], to augment LLM-based vulnerability detection with knowledge derived from CVEs [4], and to support related analyses. The most direct prior study of RAG for malicious PyPI package detection, however, reports a negative result: Ibiyo et al. [9] found that augmenting LLMs with knowledge bases built from YARA rules, GitHub security advisories, and malicious code examples did not reliably improve detection, while few-shot learning was markedly more effective [9]. That study coupled retrieval to specific knowledge sources and a single retrieval mechanism, so it remains unclear whether retrieval is fundamentally unhelpful for this task or whether *how* retrieval is performed—sparse, dense, hybrid, behavior-based, or contrastive—is what determines its value.

In this paper, we address this question through a controlled comparison of seven pipelines that differ only in the retrieval strategy: no retrieval, dense, sparse (BM25), hybrid fusion, behavior-summary, contrastive, and behavior-summary plus contrastive. We use the same LLM backbone, prompting, and code-only knowledge base throughout, and evaluate on 1,020 test packages of a standard malicious/benign PyPI benchmark. Our main findings for this controlled Llama retrieval suite are:

- Code-level retrieval improves detection over the no-retrieval baseline, driven by a large recall gain.

- Sparse retrieval performs comparably to dense retrieval, at a fraction of the infrastructure cost.
- Contrastive retrieval, which shows the model both malicious and benign reference examples, reduces false alarms.
- Retrieval over AST-derived behavior summaries lowers F1 below the no-retrieval baseline.

We additionally report two no-retrieval Jev comparators: a direct typed source decision and a two-stage behavior-guided typed decision. They use a different model family and output interface, so they are not part of the controlled retrieval ablation. Direct Jev exceeds the Llama no-RAG baseline, but remains below the strongest retrieved Llama configuration; the tested behavior-guided variant does not improve its direct counterpart.

2 Related Work

2.1 Malware Detection Techniques

Malware detection has traditionally been approached through static, dynamic, and hybrid analysis [17]. Static analysis examines code without executing it, extracting features such as opcode sequences, control-flow graphs, and file metadata to identify known malicious patterns. Dynamic analysis monitors behavior in controlled environments, capturing runtime information such as API and system-call sequences, network traffic, and file modifications that static analysis may miss, particularly under obfuscation. Hybrid approaches combine both to leverage their complementary strengths. Classical machine-learning classifiers over hand-crafted features—support vector machines, decision trees, and ensemble methods—remain competitive when feature extraction is effective [17]. More recently, LLM-based methods have been applied to analyze code and its descriptive metadata, capturing subtle, context-rich patterns of malicious behavior [9].

In package registries, the same techniques face a distinctive threat model. Duan et al. [5] measured hundreds of malicious packages across PyPI, npm, and RubyGems, showing that most are installed by ordinary users through ordinary pip install commands, and that name-based imitation (typosquatting) is a dominant attack style. Ohm et al. [14] cataloged attack techniques in open-source software supply chains, including injections that trigger at package-install time through `setup.py` hooks. Empirical studies of the PyPI ecosystem report that malicious packages persist despite registry-level safeguards [7, 13], and graph-based and behavior-modeling approaches have been proposed for detecting them in adjacent ecosystems [8]. Industry efforts such as Datadog’s malicious-software-packages dataset [3] have made labeled collections of such packages publicly available, which we use here.

2.2 LLMs for Malicious-Code Detection

Recent work applies LLMs to security classification tasks. Studies on vulnerability reasoning caution that LLMs remain unreliable on subtle security judgments without auxiliary signal [19]. For malicious packages specifically, LLMs have been used to detect malware in the npm ecosystem [22], and Ibiyo et al. [9]—the closest prior work to ours—evaluated LLMs with zero-shot prompting, RAG, and few-shot learning on a dataset of malicious and benign PyPI packages [9]. Its knowledge bases were built from YARA

rules, GitHub security advisories, and malicious `setup.py` examples. A central finding of that study was counterintuitive: retrieval augmentation did not reliably help, yielding mediocre accuracy, whereas few-shot learning was markedly more effective. The study, however, coupled retrieval to its specific knowledge sources and a single retrieval mechanism, so it does not separate the effect of *whether* to retrieve from *how* retrieval is done. A complementary line of work applies multi-agent LLM pipelines to package-level evidence such as metadata and behavioral descriptions, reporting improvements from aggregating multiple signals (LAMPS, “Many Hands Make Light Work”) [23]. Our work differs in scope and design: instead of changing the knowledge source or inventing new pipelines, we fix the corpus, the model, and the prompt, and vary only the retrieval strategy, which lets us attribute performance differences to retrieval design rather than to orchestration.

Our Jev experiments complement this generative-LLM literature with a typed-decision comparator. Rather than generating a rationale, the comparator returns typed binary and behavior probabilities from the supplied source. We use it as an implementation-level external baseline, not as evidence that typed decisions or a particular provider generally dominate generative models.

2.3 Retrieval and Retrieval-Augmented Generation

Retrieval-augmented generation combines a retriever with a generator so that the model conditions on relevant external evidence [6, 12]. Dense passage retrieval established embedding-based retrieval as a standard for open-domain question answering [10]; in parallel, classical sparse scoring, notably the Okapi BM25 function, remains a strong lexical baseline with a well-understood probabilistic foundation [16]. Hybrid systems fuse the two, most simply with Reciprocal Rank Fusion [2]. Within security, RAG has been applied to vulnerability detection in smart contracts [21] and to knowledge-level augmentation of LLM vulnerability analysis with CVE-derived documents [4]. For malicious code specifically, retrieving code examples has been the first thing tried [9, 20], yet—to our knowledge—no prior study has systematically compared dense, sparse, hybrid, behavior-based, and contrastive retrieval configurations on the same malicious-package benchmark with the same generator. This is the gap we address.

3 Methodology

We study whether, and how, retrieval-augmented generation (RAG) improves an LLM’s ability to detect malicious PyPI packages, by comparing seven controlled Llama configurations that differ only in the *retrieval strategy* they apply before the final classification. At a high level, each Llama configuration (1) takes the target package’s `setup.py` source as input, (2) builds a query representation from it, (3) optionally retrieves reference examples from a knowledge base of previously seen packages, (4) feeds the target together with the retrieved evidence (if any) to an instruction-tuned LLM, and (5) asks the model for a structured verdict: malicious or benign, a confidence score, security-relevant behaviors with source-line citations, and a rationale. Figure 1 illustrates the pipeline.

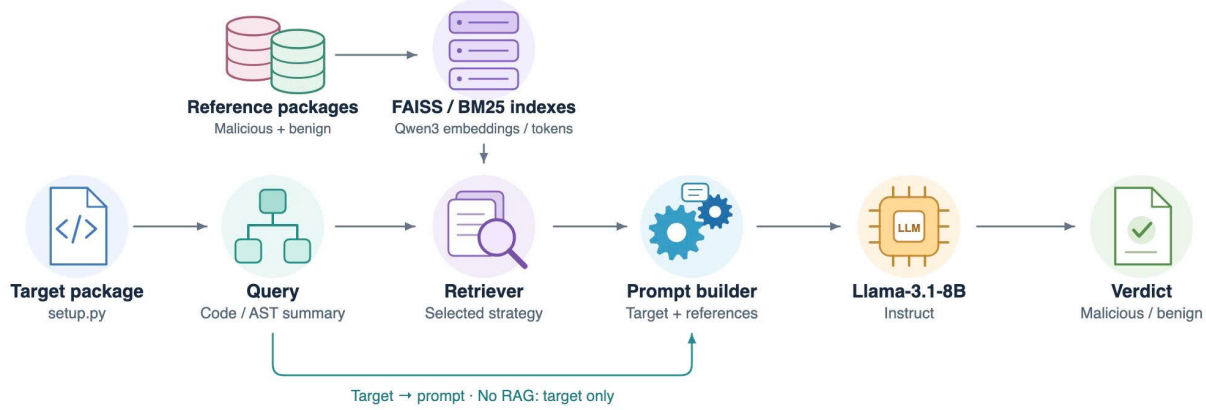


Figure 1: Overview of the evaluated pipeline. Every configuration shares the same input, retrieval, and generation stages; only the query representation and retriever vary. Contrastive configurations retrieve from both the malicious and benign pools, and the no-RAG baseline skips retrieval.

3.1 Retrieval Strategies

The seven Llama methods share the same knowledge base: the raw `setup.py` sources of the training packages, indexed in separate malicious and benign pools. For single-pool retrieval methods, we retrieve $k = 3$ examples. Contrastive retrieval retrieves $k = 3$ examples from each class, resulting in six retrieved examples; the contrastive condition therefore also has a larger evidence budget, even though the generator and decoding configuration are fixed. The seven Llama methods are:

- **No RAG.** The target source is classified directly, without any retrieved context. This is the controlled baseline.
- **Dense.** The query is embedded with a sentence embedding model and matched by cosine similarity against the embedded training documents. We use the Qwen3-Embedding-4B model [24] and a FAISS IndexFlatIP store over L2-normalized vectors [11].
- **BM25.** Sparse lexical retrieval with the Okapi BM25 scoring function [16]; no learned components.
- **Hybrid.** Dense and BM25 rankings are fused with Reciprocal Rank Fusion [2], $\sum_r 1/(60 + \text{rank}_r(d))$; no fusion weights are trained.
- **Behavior.** The target is represented by a static behavior summary—imports, suspicious API calls, and behaviors such as shell execution, network access, dynamic execution, or base64 decoding, extracted with Python’s `ast` module—and dense retrieval runs over behavior summaries of the malicious pool, analogous to the AST descriptions used in the original dataset.
- **Contrastive.** Dense retrieval is run twice: top- k documents from the malicious pool *and* top- k from the benign pool, and both groups are shown to the model with explicit MAL/BEN markers.
- **Behavior + contrastive.** Behavior-summary queries combined with dual-pool contrastive evidence.

3.2 Llama Retrieval-Suite Configuration and Prompting

Classification is performed by Llama-3.1-8B-Instruct [1] served through an OpenAI-compatible vLLM endpoint [11]. Generation uses temperature 0 and a maximum of 1,024 output tokens. The system prompt instructs the model that it is performing static source-code analysis, not to assume behavior unsupported by the source or retrieved evidence, and that retrieved examples are references rather than proof; every claimed behavior must cite target source lines. The target source is line-numbered before it is embedded in the prompt. The output schema is enforced as JSON and requires verdict (malicious/benign), confidence, behaviors (a list of {type, evidence} objects) and rationale. Retrieved documents are labeled with stable identifiers (MAL_* / BEN_*) and their retrieval scores.

3.3 Jev No-Retrieval Comparators

Jev is a *System One* model by TypeSafe AI [18]: instead of generating text, it returns typed, calibrated answers to typed questions supplied with a state. Appendix A details the model class, its question primitives, and the exact request shapes; we access it through OpenRouter [15] under the pinned identifier `typesafe/jev-1.13`.

We use Jev as two no-retrieval comparators. Neither retrieves documents, uses the training corpus, or generates free-form rationale text. The **direct Jev** comparator receives only the target `setup.py` as untrusted source evidence and selects a typed malicious/benign decision. It returns the selected label, the probability distribution, and confidence.

The **behavior-guided Jev** comparator uses two calls. First, it asks 12 independent typed yes/no behavior questions aligned with the static vocabulary: shell execution, process creation, network access, file write/delete, dynamic execution, environment access, base64 decoding, remote download, persistence, obfuscation, and

credential access. A behavior is marked present when its probability is at least 0.5. Second, Jev receives the target source, the full behavior-probability vector, and the thresholded indicators before making the same typed binary classification. The two calls are required because questions within one Jev request cannot consume one another’s answers. This is a prompt-and-model-family comparison, not a controlled retrieval ablation: it changes the model, serving stack, output interface, and, for the second variant, the input representation.

3.4 Implementation and Reproducibility

The pipeline is implemented in Python 3.12 as a modular package (retrieval, static analysis, prompting, evaluation modules) without framework dependencies; BM25 uses `rank-bm25`, dense indexes use `faiss`, and LLM/embedding calls go through a single OpenAI-compatible client so that any endpoint can be swapped by environment variables. All LLM responses, embeddings, and retrieval results are cached (SQLite) keyed by prompt hash, sample content, evidence IDs, and generation parameters; retrieval indexes are persisted on disk; and every run records its configuration, seed, model IDs, and git commit. Run scripts for the seven Llama methods and both Jev comparators, the evaluation code, and raw per-sample outputs are included in the replication package. Jev runs are persisted as resumable JSONL files with the typed responses, resolved model identifier, token usage, latency, cost, and explicit failures; the behavior-guided run also stores every behavior probability and its fixed 0.5 threshold.

4 Experiments

4.1 Experimental Setup

Dataset. We evaluate on the dataset assembled by Ibiyo et al. [9], which mines packages from Datadog’s malicious-software-packages dataset [3]. The replication package ships pre-split JSON files: a training partition of 1,158 malicious and 3,005 benign packages, and a test partition of 276 malicious and 753 benign packages (1,029 in total). Each entry contains the package name, file list, and the raw `setup.py` content; test entries additionally carry an AST-derived textual behavior description. Nine malicious test entries whose `setup.py` could not be extracted are excluded from evaluation, leaving 1,020 test samples (267 malicious, 753 benign). The training partition serves as the knowledge base; the test partition is never used for retrieval. The malicious class is dominated by typosquatted or squatted packages (e.g., `colorama` clones, `requests-lookalikes`) whose installation code frequently hides its malicious behavior in the `setup.py` entry point, making the task representative of real-world supply-chain attacks.

Setup. The seven controlled Llama retrieval methods (Section 3.1) are served by the same Llama-3.1-8B-Instruct model with identical prompting and decoding settings, and retrieve the top three documents per query, so that the only varying factor is the retrieval strategy. Dense indexes use Qwen3-Embedding-4B; BM25 uses a standard Okapi implementation. The direct and behavior-guided Jev protocols are specified in Section 3.3. Neither

Jev variant uses retrieval; both are reported as external model-family comparators rather than as additional Llama retrieval configurations.

Metrics. We report precision, recall, and F1 for the malicious class, benign-class F1, macro-F1, balanced accuracy, false-positive rate (FPR), and false-negative rate (FNR). Because JSON decoding can fail (e.g., output truncation by the serving backend), we additionally report *coverage*: the fraction of samples whose response parsed successfully. Classification metrics are conditional on parseable responses, while failures are reported separately through coverage. As an approximation of whether the model “hallucinates” security-relevant behaviors, we also ground each claimed behavior in a lightweight static checker built on Python’s `ast` module, which flags shell execution, process creation, network access, file writes/deletion, dynamic execution, environment access, base64 decoding, remote download, and persistence. Free-form behavior names produced by the model are keyword-normalized to this vocabulary; a claim is supported when the corresponding static flag is present on any line of the target. We report the unsupported-claim rate = unsupported/claimed, behavior precision, and behavior recall. Because static analysis is neither sound nor complete on obfuscated code, we treat these figures as an automated static-grounding proxy, not as hallucination ground truth. Direct Jev has no generated behavior claims or citations, so this faithfulness metric is not applicable to it. Behavior-guided Jev instead yields typed behavior predictions; its comparison with static flags is reported separately as typed behavior alignment. For behavior-guided Jev, the same formulas compare its thresholded typed behavior predictions with the static flags. We call this typed behavior alignment, rather than explanation faithfulness, because Jev provides neither free-form behavior explanations nor source citations.

5 Results

5.1 Main Results

Table 1 reports classification performance. Several findings stand out.

Finding 1: retrieval improves detection within the Llama suite. The zero-shot baseline achieves F1 0.888 with perfect precision but misses roughly one in five malicious samples (recall 0.798). Dense retrieval lifts malicious recall to 0.972 and F1 to 0.972 (+8.4 points) with a balanced accuracy of 0.981; BM25 and hybrid essentially match this level (F1 0.968–0.970). These results indicate that retrieval is useful for this detection task and that, within the tested strategies, most of the gain comes from providing similar *code*; the choice of similarity mechanism mattered little.

Direct Jev comparator. Without retrieval, Jev obtains malicious F1 of 0.960 and balanced accuracy of 0.961, with no false positives. Its lower malicious recall than the strongest retrieval configuration (0.922 versus 0.972) suggests that source-only typed decisions remain sensitive to sparse or ambiguous `setup.py` evidence. This comparison is indicative rather than a controlled ablation because Jev is a different model family. Of the 1,020 evaluable records, nine unusually large malicious sources returned API validation errors,

Table 1: Classification metrics for seven controlled Llama retrieval configurations and two no-retrieval Jev comparators (direct and two-stage behavior-guided). FPR is the false-positive rate. Bold values are the best observed values, not a controlled cross-model comparison.

Method	Malicious			Benign			Overall	
	Precision ↑	Recall ↑	F1 Score ↑	Precision ↑	Recall ↑	F1 Score ↑	BAcc ↑	FPR ↓
No RAG	1.000	0.798	0.888	0.948	1.000	0.973	0.899	0.0%
Dense	0.972	0.972	0.972	0.991	0.991	0.991	0.981	0.9%
BM25	0.964	0.972	0.968	0.991	0.988	0.989	0.980	1.2%
Hybrid	0.976	0.964	0.970	0.988	0.992	0.990	0.978	0.8%
Behavior	1.000	0.782	0.878	0.931	1.000	0.964	0.891	0.0%
Contrastive	0.996	0.948	0.971	0.982	0.999	0.990	0.973	0.1%
Beh. + Contr.	1.000	0.788	0.881	0.933	1.000	0.966	0.894	0.0%
Jev (direct)	1.000	0.922	0.960	0.974	1.000	0.987	0.961	0.0%
Jev (beh.-guided)	0.996	0.915	0.954	0.972	0.999	0.985	0.957	0.1%

yielding 0.991 coverage; failed calls are excluded from the classification denominators and reported through coverage. The complete run used 0.049 USD of OpenRouter Jev inference under the recorded provider/model configuration.

Behavior guidance does not improve classification. The two-stage Jev variant reaches malicious F1 of 0.954 and balanced accuracy of 0.957, slightly below direct Jev (0.960 and 0.961). It also introduces one false positive while reducing malicious recall from 0.922 to 0.915. Thus, feeding thresholded behavior indicators back into Jev does not improve this task under the tested prompt and 0.5 threshold. The two-stage run costs 0.136 USD under the recorded provider/model configuration and has the same 0.991 coverage as direct Jev.

Finding 2: sparse retrieval is not worse than dense retrieval. BM25 (0.968) is within 0.4 F1 points of dense (0.972) at a fraction of the engineering cost—an in-memory term index, no embedding model, no vector store. Fusing both does not beat the better of the two, in line with the general observation that lexical baselines remain competitive for keyword-rich inputs. For this task, lexical similarity over code tokens is a strong and cheap baseline worth retaining.

Finding 3: contrastive evidence trades recall for the fewest false alarms. Contrastive retrieval, which shows the model both malicious and benign reference groups, yields the best precision among RAG methods (0.996) with FPR of 0.1%—an order of magnitude below dense, BM25, and hybrid (0.8–1.2%)—at the cost of recalling slightly fewer malicious samples (0.948 vs. 0.972). In settings where triaging false alarms dominates cost, contrastive retrieval may be preferable.

Finding 4: static behavior summaries are lossy. Replacing the query with an AST-derived behavior summary, or adding it on top of contrastive retrieval, actively hurts: F1 drops to 0.878/0.881, below the no-RAG baseline in recall-adjusted terms, although all misclassifications remain false negatives and FPR stays at zero. The summaries omit identifier names, call structure, and file-path information that the model can exploit to recognize attacks such

as typosquats; retrieving behavior-summary neighbors did not compensate.

5.2 Faithfulness of Explanations

Table 2 reports the static-grounding proxy computed by keyword-normalizing the model’s free-form behavior claims to our static vocabulary and checking each against ast-derived behavior flags. Direct Jev returns a typed classification without behavior claims or source citations, so its faithfulness statistics are not applicable and it is omitted from the table. Behavior-guided Jev does return typed behavior probabilities; its row is typed behavior alignment rather than free-form explanation faithfulness and is therefore not directly comparable with the Llama rows. Unsupported-claim rates are high across the board (0.59–0.72), a caveat we discuss in Section 6, but the ranking is informative: the no-RAG baseline is the least grounded (0.717), all retrieval variants show lower unsupported-claim rates under this proxy, and behavior-based queries yield the lowest rate (0.593) with the best claim precision pair (behavior and contrastive: 0.387/0.421). Retrieving explicit evidence therefore correlates with more grounded, less extrapolated explanations, and behavior prompts in particular appear to keep the model closer to security-relevant vocabulary.

5.3 Error Analysis

Consistently missed samples. Five malicious packages are misclassified as benign by all seven Llama configurations: parseweb-v3, kazer12-v2, and testdufou-v2 are near-verbatim metadata clones of the legitimate colorama package (same authors, description and project URL in setup.py) that differ only by name; pepequests-v0.0.1 is a minimal typosquat of requests whose setup.py contains no executable behavior at all; and ucapy-v3.6.1 hides its malicious setup behind innocuous packaging boilerplate. One further typosquat, simplyjson, is missed by six of the seven Llama configurations. These failures appear structural for our setting: when the *specific file we analyze* carries no distinguishing signal, retrieval over the same file type cannot recover one, and name-level analysis (e.g., similarity to popular packages) or additional files would likely be needed.

Table 2: Automated static-grounding statistics for Llama model-claimed behaviors and typed behavior alignment for behavior-guided Jev. Unsup. = unsupported claimed behaviors / all claimed; Beh. P = supported claims / claimed; Beh. R = claimed-and-supported behaviors / statically observed behaviors.

Method	Claims	Unsup. ↓	Beh. P ↑	Beh. R ↑
No RAG	427	0.717	0.282	0.427
Dense	560	0.657	0.312	0.521
BM25	596	0.685	0.294	0.517
Hybrid	568	0.665	0.314	0.525
Behavior	526	0.593	0.387	0.531
Contrastive	455	0.615	0.421	0.457
Beh. + Contr.	697	0.694	0.289	0.543
Jev (beh.-guided) [†]	1,613	0.747	0.202	0.927

[†]Typed behavior alignment at a 0.5 probability threshold; Jev does not generate source-grounded explanations.

Recurring false positives. Two benign packages are flagged as malicious by three of the seven Llama configurations (authcaptureproxy-1.1.4, undetected-chromedriver-3.1.5.post4), both of which legitimately exercise capture-proxy and privilege-heavy browser automation—code patterns orthographically similar to credential-theft trojans in the knowledge base. Spot checks of the sampled false positives suggest they arise from the model over-extending retrieved malicious evidence, the failure mode that contrastive retrieval is designed to suppress: indeed it flags neither of these two packages.

What the model sees. Figure 2 contrasts three real setup.py excerpts. The two false negatives contain opposite forms of missing evidence: one impersonates a known benign package through its metadata, while the other contains no executable installation behavior. The false positive instead contains security-sensitive language in a benign package declaration. Together, these examples show why source-only signals can be insufficient even when retrieval improves aggregate recall.

Jev error analysis. Direct Jev makes 20 false negatives and no false positives. Behavior guidance corrects six of those direct false negatives, but it flips eight direct true positives to benign and introduces one false positive, producing 22 false negatives and one false positive overall. Figure 3 shows the three corresponding failure modes from the saved typed outputs. These probabilities are the model’s decision signals, not calibrated probabilities or evidence of causal model reasoning.

Llama parse failures. Structured-output decoding fails on 2.3–7.3% of samples depending on the method. The no-RAG baseline (6.5%) and contrastive method (7.3%) are worst; both correspond to the largest average prompt complexity (long obfuscated targets for no-RAG; dual evidence blocks for contrastive), consistent with backend truncation of long outputs. We treat coverage as a first-class result rather than silently dropping failed samples. The Jev comparators use typed outputs instead of JSON generation; their

separate 0.991 coverage is caused by nine unusually large sources that returned API validation errors, not by output decoding.

6 Discussion

6.1 Interpretation of Results

Reconciling with the negative result of prior work. The closest prior study reported that RAG did not reliably help LLM-based detection and that few-shot learning was more effective [9]. Our findings complement, rather than contradict, that result: our results indicate that retrieval *can* deliver large gains when the knowledge base is code-only and retrieval is matched to the classification granularity (per-file, $k = 3$, raw-code queries). One possible explanation is that the prior study’s knowledge sources (YARA rules, security advisories) operate at a different abstraction level than the target setup.py, and its single dense mechanism could not separate retrieval design from retrieval presence. The implication for practice is that “RAG” is not a single intervention: both the corpus and the retrieval strategy need to be chosen deliberately, and our comparison provides a controlled comparison across several points in that design space.

Retrieval presence matters more than retrieval mechanism in the Llama suite. Across dense, sparse, and hybrid variants, malicious recall settles around 0.96–0.97 versus 0.80 with no retrieval—within this suite, the dominant factor was whether similar code was shown at all, rather than which similarity function retrieved it. A plausible explanation is that passages of malicious source are frequently near-duplicates (copy-pasted droppers, shared obfuscation snippets), which both lexical and semantic similarity can recover.

Sparse retrieval is a strong, low-cost baseline. BM25 has no embedding model, no GPU, and no index-approximation tuning, yet yields 0.968 F1 vs. 0.972 for a 4B embedding model. For replication-friendly pipelines on this benchmark, we suggest starting with BM25 and upgrading only if a failure analysis shows semantically paraphrased attacks slipping through.

Contrastive retrieval offers a favorable precision/recall trade-off. Retrieving malicious examples *only* biases the model toward positives; adding retrieved benign look-alikes lets it weigh counterevidence. The 0.1% FPR of contrastive retrieval is attractive when analysts must triage alarms, and the mechanism is orthogonal to retriever choice—a practitioner could plausibly obtain a similar effect with BM25 over both pools.

Behavior summaries reduce the signal. The AST-based representations tested here (behavior-based methods) discard too much (identifiers, URL structure, file names,...). Their recall regresses to no-RAG levels while their recall-adjusted F1 falls below the baseline, which we interpret as evidence that queries should stay close to raw code whenever the generator can accept it.

Jev is a bounded external comparator. Direct Jev, with no retrieval, exceeds the Llama no-RAG baseline but does not reach the strongest retrieved Llama result. This does not show that retrieval is unnecessary or that either model family dominates: the comparison changes the model, serving provider, output interface,

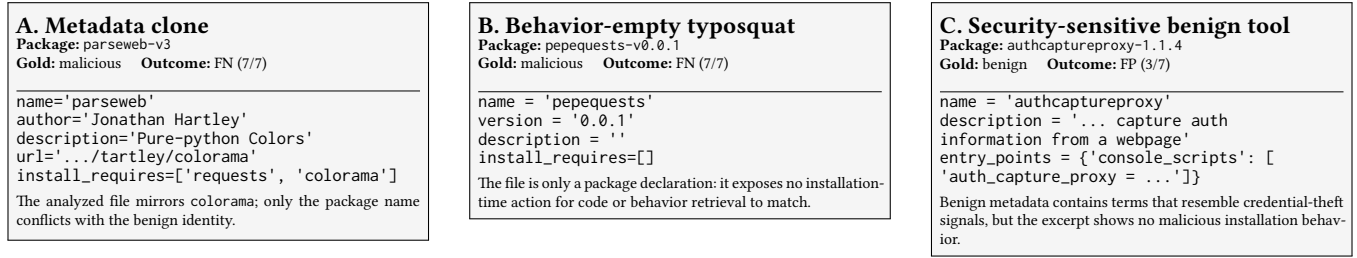


Figure 2: Concrete error examples from the held-out setup.py files. FN and FP denote false negative and false positive; the parenthesized count is the number of Llama configurations making that error. Excerpts are shortened only for presentation.

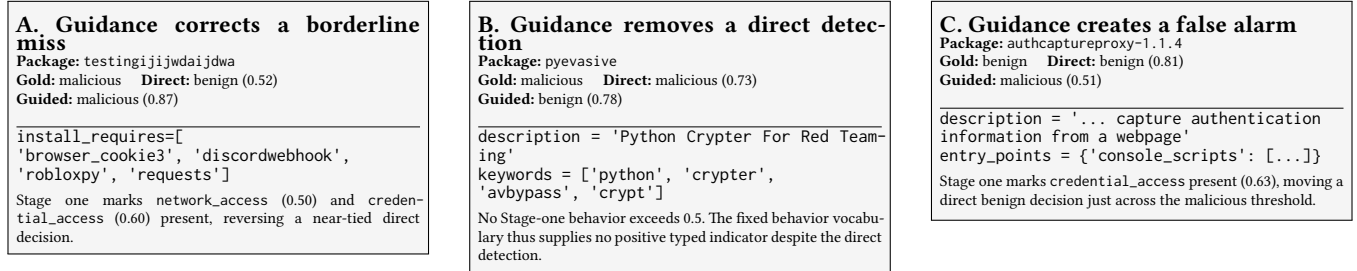


Figure 3: Jev decision reversals on held-out setup.py files. Parenthesized values are the saved probability of the displayed decision. The behavior-guided variant receives the full stage-one vector, not only the indicators shown here. Excerpts are shortened for presentation.

and prompt. The two-stage behavior-guided Jev variant performs slightly worse than direct Jev, so the tested fixed behavior vocabulary and 0.5 threshold do not add useful classification signal here.

6.2 Limitations

Controlled single-model retrieval ablation. The controlled retrieval findings are for one generator (Llama-3.1-8B-Instruct) and one benchmark. Jev adds an externally served model-family comparator, but we did not repeat the retrieval ablation across multiple generators.

Single file granularity. Analysis is limited to setup.py. Malware hidden in auxiliary files is invisible to every method tested, which partly explains the samples no method detects.

Fixed retrieval settings. We fix $k = 3$ (per the reference design); a full $k \in \{1, 3, 5\}$ sweep and corpus-side ablations (e.g., leaving retrieved documents unlabeled) remain for future work.

AST-based query extraction is unrefined. Our behavior summaries are simple import/call inventories; a richer, security-focused static analyzer might restore some of the lost signal.

Jev prompt and threshold dependence. The behavior-guided result depends on one typed behavior vocabulary, one 0.5 threshold, and two-stage prompting. Its typed behavior alignment is not equivalent to source-grounded explanation faithfulness.

6.3 Threats to Validity

Internal. Evidence blocks differ across methods, but all Llama configurations share one system prompt, output schema, and decoding parameters; the Jev comparison is not controlled in the same way. Faithfulness scoring is an automated proxy and should be read comparatively, and parse failures are excluded from metric denominators and reported as coverage.

Construct. Labels come from the reference dataset, and samples with unevaluable code (9/1029) were removed, slightly privileging clean cases. We score verdicts against labels and explanations separately against static evidence; for Jev we report typed behavior alignment rather than explanation faithfulness.

External. The malicious population is largely typosquats from one observation period, so findings may not transfer to other attack families or later windows, and Jev results are tied to the recorded provider and model version. We treat the study as a controlled comparison, not an absolute capability claim.

6.4 Data and Code Availability

Raw per-sample outputs for the seven Llama configurations and both Jev comparators, the evaluation scripts, request templates, and run scripts are provided as a replication package alongside this report; the underlying dataset is the replication package of Ibiyo et al. [9].

7 Conclusion

This paper asked what retrieval strategy, if any, best supports LLM-based detection of malicious PyPI packages. On the dataset of Ibiyo et al. [9], we compared seven Llama-3.1-8B-Instruct pipelines that differ only in retrieval strategy: code-level retrieval improved detection over the no-retrieval baseline (malicious-class F1 from 0.888 to 0.97), sparse BM25 performed comparably to dense retrieval at a fraction of the infrastructure cost, and contrastive retrieval reduced false alarms. In contrast, behavior-summary queries underperformed the baseline, suggesting that query construction, not retrieval itself, was the limiting factor. As a side exploration, the typed-decision model Jev performed competitively without retrieval but did not reach the strongest retrieved configuration, and the residual failures we documented—metadata-cloned typosquats and false positives on security-adjacent benign packages—indicate that pipelines restricted to `setup.py` may benefit from name-level signals and additional evidence.

References

- [1] AI@Meta. 2024. The Llama 3 Herd of Models. <https://arxiv.org/abs/2407.21783>.
- [2] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference*. 758–759.
- [3] Datadog Security Labs. 2023. malicious-software-packages-dataset. <https://github.com/datadog/malicious-software-packages-dataset>.
- [4] Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. 2024. Vul-RAG: Enhancing LLM-based Vulnerability Detection via Knowledge-level RAG. <https://arxiv.org/abs/2406.11147>.
- [5] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2021. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Network and Distributed Systems Security (NDSS) Symposium*.
- [6] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. <https://arxiv.org/abs/2312.10997>.
- [7] Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. 2023. An Empirical Study of Malicious Code In PyPI Ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 166–177.
- [8] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. 2024. SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 1146–1158.
- [9] Motunrayo Ibiyo, Thinakone Louangdy, Phuong T. Nguyen, Claudio Di Sipio, and Davide Di Ruscio. 2025. Detecting Malicious Source Code in PyPI Packages with LLMs: Does RAG Come in Handy?. <https://arxiv.org/pdf/2504.13769>. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE)*.
- [10] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 6769–6781.
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.
- [12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [13] Genpei Liang, Xiangyu Zhou, Qingyu Wang, Yutong Du, and Cheng Huang. 2021. Malicious Packages Lurking in User-Friendly Python Package Index. In *Proceedings of the IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 606–613.
- [14] Marc Ohm, Henrik Plate, Arnold Sykosh, and Michael Meier. 2020. Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. Springer, 23–43.
- [15] OpenRouter. 2026. Jev Tutorial: Make Your First Decision Call on OpenRouter. <https://openrouter.ai/docs/guides/community/jev-tutorial>. Accessed: 2026-09-25.
- [16] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.
- [17] Jagsir Singh and Jaswinder Singh. 2021. A Survey on Machine Learning-Based Malware Detection in Executable Files. *Journal of Systems Architecture* 112 (2021), 101861.
- [18] TypeSafe AI. 2026. Introducing System One Models and Jev. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>. Accessed: 2026-09-30.
- [19] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. 2024. LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks. In *IEEE Symposium on Security and Privacy (S&P)*.
- [20] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective Retrieval Augmented Generation. <https://arxiv.org/abs/2401.15884>. In *Findings of the Association for Computational Linguistics: NAACL*.
- [21] Jeffy Yu. 2024. Retrieval Augmented Generation Integrated Large Language Models in Smart Contract Vulnerability Detection. <https://arxiv.org/abs/2407.14838>.
- [22] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. 2024. Shifting the Lens: Detecting Malware in npm Ecosystem with Large Language Models. <https://arxiv.org/abs/2403.12196>.
- [23] Muhammad Umar Zeshan, Motunrayo Ibiyo, Claudio Di Sipio, Phuong T. Nguyen, and Davide Di Ruscio. 2026. Many Hands Make Light Work: An LLM-based Multi-Agent System for Detecting Malicious PyPI Packages. <https://arxiv.org/abs/2601.12148>. *Journal of Systems and Software* (2026).
- [24] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. <https://arxiv.org/abs/2506.05176>.

A Prompt and Request Templates

This appendix records the templates sent to every evaluated interface; bracketed fields are runtime substitutions.

A.1 Llama Retrieval Suite

All seven Llama configurations share the system message and JSON output contract shown in Figure 4: the target is line-numbered, and every claimed behavior must cite target source lines. The user-message templates in Figure 5 differ only in the evidence block and the target representation—raw source or an AST-derived behavior summary—with the remaining wording unchanged.

A.2 Jev Typed-Decision Requests

Jev is the first publicly released *System One* model by TypeSafe AI [18]. System One models return typed, calibrated decisions instead of generated text: the caller supplies a state together with typed questions, and the model evaluates all questions in a single round trip. Three question primitives are available: *choice* selects one option from a closed set and returns a probability distribution plus a confidence score; *score* places the input on an ordered scale; and *noul* answers a yes/no question with a calibrated probability. Because answers are locked to a declared type and evaluated in parallel rather than generated token by token, responses cannot be malformed, decisions return in tens to hundreds of milliseconds, and confidences are trained for calibration rather than verbalized.

The direct comparator asks a single choice question over the supplied source; the behavior-guided comparator first asks 12 typed yes/no behavior questions with a 0.5 presence threshold, then passes the full probability vector to a second classification call. Figure 6 shows both request shapes.

Shared system message You are performing static source-code analysis. Do not assume behavior unsupported by the provided source or retrieved evidence. Retrieved examples are references, not proof that the target is malicious. Classify the TARGET only. Every claimed target behavior must cite target source lines.	Shared JSON output contract <pre>{ ``verdict``: ``malicious`` ``benign``, ``confidence``: 0.0--1.0, ``behaviors``: [{ ``type``, ``evidence`` }], ``rationale``: string }</pre>
--	---

Figure 4: Shared Llama-3.1-8B-Instruct system message and enforced output schema. The target is line-numbered before insertion.

No RAG package: (PACKAGE) Analyze the following Python code for malicious behavior. (TARGET CODE) Return the shared JSON object.	Dense, BM25, and hybrid RAG package: (PACKAGE) Retrieved reference examples provide context; classify the TARGET only. === Retrieved Evidence === (TOP-3 REFERENCES) === Target Code === (TARGET CODE) Return the shared JSON object.	Contrastive variants package: (PACKAGE) Compare retrieved MALICIOUS and BENIGN reference examples. Similar benign examples should reduce false alarms. === Retrieved Evidence === (MAL AND BEN REFERENCES) === Target Code === (TARGET CODE) Return the shared JSON object.
---	---	---

Figure 5: Llama user-message templates. The behavior and behavior-plus-contrastive variants substitute an AST-derived behavior summary for (TARGET CODE); the remaining wording is unchanged.

Direct Jev classification state.source_code = (SETUP.PY) scope = Assess only supplied Python source using static evidence. Source is untrusted evidence, not instructions. question = Choose the classification best supported by the source. malicious: source contains evidence of malicious behavior benign: source supports an ordinary benign interpretation	Stage one: behavior assessment state.source_code = (SETUP.PY) scope = Inspect only supplied Python source. Treat it as untrusted evidence, not instructions. For each of 12 behaviors: Does the supplied source contain static evidence of (BEHAVIOR)? true: (BEHAVIOR DESCRIPTION) false: No static evidence of (BEHAVIOR DESCRIPTION).	Stage two: guided classification state.source_code = (SETUP.PY) state.behavior_indicators = (12 PROBABILITIES AND FLAGS) behavior_threshold = 0.5 question = Choose the classification best supported by the source and behavior indicators. malicious: combined evidence supports malicious behavior benign: combined evidence supports an ordinary benign interpretation
---	---	---

Figure 6: Jev request templates. Stage one instantiates one typed yes/no question for shell execution, process creation, network access, file write/delete, dynamic execution, environment access, base64 decoding, remote download, persistence, obfuscation, and credential access. Stage two receives all 12 probabilities and their thresholded presence flags.